

Information Retrieval System Report

Overview

This report documents a Streamlit-based information retrieval application built around the CISI collection and a companion dataset-preparation script. The application supports dataset inspection, preprocessing analysis, inverted-index construction, and keyword-based retrieval over CISI documents.

The submission also includes repository proof, deployed application proof, dataset files, and screenshots that demonstrate the application running in the virtual lab environment.

Submission requirements

The provided requirement image states that the submission should include Streamlit application code, the dataset used, a report explaining the implementation in the virtual lab, screenshots of the Streamlit front end, experimental results with tables and inferences, demo evidence, and a README section describing dependency installation and the command used to run the app.

This project satisfies those components through the Python application file, supporting dataset-preparation script, CISI dataset CSV files, this report, GitHub repository proof, deployed Streamlit proof, and multiple interface screenshots showing different parts of the system in operation.

Repository and deployment

The source repository for the assignment is available at [GitHub Repository](#), and the attached repository screenshot provides submission proof that the codebase has been uploaded online.

The deployed application is available at [Streamlit Application](#), and the attached screenshots provide visual proof of the live interface and its major modules.

Application structure

The Streamlit interface defines six tabs: Dataset & Overview, Text Preprocessing, Phrase Queries, Dictionary Search, Tolerant Retrieval, and Results & Inference. The visible application code shows that the system is designed as a teaching-oriented IR lab where users can upload a CSV dataset, inspect documents, preprocess text, build an inverted index, and issue keyword searches.

The `prepare_dataset.py` script converts CISI source files into `cisi_docs.csv`, `cisi_queries.csv`, and `cisi_rel.csv`, separating documents, queries, and relevance judgments into clean tabular resources for

experimentation. The resulting files provide the data foundation for both interactive retrieval and future benchmark-style evaluation.

Preprocessing and retrieval

The preprocessing pipeline uses regex tokenization together with optional lowercasing, stopword removal, hyphen normalization, stemming, and lemmatization. NLTK resources for stopwords and WordNet are downloaded at runtime, and the interface exposes the normalization controls through selectable options.

The inverted index maps tokens to document IDs, and search is performed using conjunctive intersection over query-term postings lists. As a result, the current implementation behaves as an AND-based retrieval model rather than a ranked search engine.

Screenshots and evidence

The attached screenshots provide evidence for the key parts of the submitted system.

Evidence item	What it shows
GitHub.jpg	Repository proof showing the uploaded assignment files on GitHub.
P1.jpg	Dataset loader and file-system entry screen with dataset preview and summary metrics.
P2.jpg	Lexical normalization pipeline with raw text, standard tokens, stemmed tokens, and lemmatized tokens.
P3.jpg	Multi-word phrase query interface with biword and positional proximity matches.
P4.jpg	Performance benchmark screen comparing BST and B-Tree dictionary search latency.
P5.jpg	Error fault-tolerance verification interface showing expanded internal term handling.
P6.jpg	End-to-end inference and structural analysis screen with observations and conclusions.
image-8.jpg	Requirement sheet listing submission components, demo evidence, and README expectations.

These screenshots help satisfy the requirement for front-end proof, demo evidence, and experimental-result presentation in the report.

Data assets

The attached `cisi_docs.csv` file contains document records with `doc_id`, `title`, and `text` fields from the CISI collection. The attached `cisi_queries.csv` file stores natural-language information needs, and `cisi_rel.csv` stores query-document relevance pairs for evaluation use.

This separation of corpus data, query data, and relevance labels is appropriate for an introductory IR assignment because it supports both retrieval demonstrations and metric-based validation.

How to run the app

The requirement sheet explicitly asks for dependency installation steps and the command used to run the Streamlit application. Based on the application imports, the local environment should include Python together with Streamlit, pandas, and NLTK.

A practical run procedure is:

1. Place `app.py`, `prepare_dataset.py`, and the dataset CSV files in the same project folder, or clone the GitHub repository into a local workspace.
2. Install dependencies with `pip install streamlit pandas nltk`.
3. Start the application with `streamlit run app.py`, which matches the requirement example for the README section.
4. When the browser opens, upload the CISI CSV file in the dataset tab, inspect preprocessing options, build the inverted index, and run sample keyword queries through the interface.

Experimental results

The dataset preview screenshot shows the structured CISI document table loaded into the application, with the columns `doc_id`, `title`, and `text` visible in the interface. This confirms that the uploaded dataset is being read successfully and presented in tabular form before further preprocessing and retrieval steps.

Dataset preview table

The following table reproduces the visible sample rows from the attached screenshot.

Row	doc_id	title
0	1	18 Editions of the Dewey Decimal Classifications
1	2	Use Made of Technical Libraries
2	3	Two Kinds of Power An Essay on Bibliographic Control
3	4	Systems Analysis of a University Library; final report and research project
4	5	A Library Management Game: a report on a research project
5	6	Abstracting Concepts and Methods

6	7	Academic Library Buildings A Guide to Architectural Issues and Solutions
7	8	The Academic Library Essays in Honor of Guy R. Lyle
8	9	Access to Libraries in College
9	10	Access to Periodical Resources

Inference from the table

This table demonstrates that the system is loading document identifiers, titles, and full-text content into a searchable dataframe structure, which is appropriate for downstream indexing and retrieval experiments. It also supports the report requirement for including tables and experimental evidence from the Streamlit front end.

Strengths

Several strengths are clear from the code organization and the submitted evidence.

- The project separates dataset preparation from the interactive application, which improves maintainability and clarity.
- The interface makes preprocessing choices visible, which is useful for explaining token normalization and retrieval behavior.
- The screenshots indicate that the broader submission includes modules for phrase queries, benchmarking, fault tolerance, and inference presentation beyond the minimal visible code excerpt.
- The repository and deployed link provide strong submission proof for both source code and application availability.

Limitations and improvements

Some limitations are still visible in the attached `app.py` excerpt. The shown retrieval logic is conjunctive and unranked, and the visible index-building loop inserts raw tokens even when stemming or lemmatization options are enabled.

Recommended next steps are to align indexing with the selected normalization mode, add ranking such as TF-IDF or BM25, and integrate `cisi_queries.csv` with `cisi_rel.csv` for quantitative evaluation metrics such as precision and recall.

Screenshots:

Screenshots of execution in virtual lab.

This screenshot shows the first step of a data processing pipeline. The interface includes a sidebar with 'Engine Global Controls' and a main content area titled '1. Dataset Loader & File System Entry'. A green status bar indicates 'System fully populated with 1000 records from distributed source(s)'. Below this, there are two sections: 'Data Matrix Head Overview' and 'Statistical Metric Distribution'. The 'Data Matrix Head Overview' table lists 10 rows of data with columns for 'id', 'text', and 'score'. The 'Statistical Metric Distribution' section shows a 'Document Population' of 174,025 and a 'Mean Document Word Count' of 119.20 words.

id	text	score
0	Introduction of the theory, background, and objectives.	The present study is a history of the 2007 Recital (Class) Project.
1	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
2	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
3	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
4	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
5	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
6	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
7	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
8	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
9	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
10	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.

This screenshot shows the second step of the data processing pipeline, titled '2. Lexical Normalization Pipeline Insights'. It features a sidebar with 'Engine Global Controls' and a main content area. A green status bar indicates 'System fully populated with 1000 records from distributed source(s)'. Below this, there are three sections: 'Source Document Raw Payload', 'Standard Tokens Pipeline', and 'Porter Stemmer Stream'. The 'Source Document Raw Payload' section shows a list of documents. The 'Standard Tokens Pipeline' and 'Porter Stemmer Stream' sections show the results of tokenization and stemming, respectively. The 'WordNet Lemmatizer Stream' section shows the results of lemmatization.

id	text	score
0	Introduction of the theory, background, and objectives.	The present study is a history of the 2007 Recital (Class) Project.
1	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
2	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
3	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
4	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
5	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
6	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
7	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
8	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
9	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
10	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.

This screenshot shows the third step of the data processing pipeline, titled '3. Multi-word Phrase Queries Extraction Mechanism'. It features a sidebar with 'Engine Global Controls' and a main content area. A green status bar indicates 'System fully populated with 1000 records from distributed source(s)'. Below this, there are two sections: 'Biword Index Matches' and 'Positional Proximity Matches'. The 'Biword Index Matches' section shows a list of biword matches. The 'Positional Proximity Matches' section shows a list of positional proximity matches.

id	text	score
0	Introduction of the theory, background, and objectives.	The present study is a history of the 2007 Recital (Class) Project.
1	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
2	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
3	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
4	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
5	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
6	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
7	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
8	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
9	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.
10	Our first of technical details.	The report is a collection of 100 words of words in the Recital (Class) Project.

Rocky LinuxRocky WSLRocky ForumRocky MailmanRocky Redd

Engine Global Controls

- Indexing
- Document Removal
- Hybrid Ranking

Report IssuesFeedback ComparisonPhrase ProcessingDictionary ToolsDocument RemovalIndexing Log

6. End-to-End System Inferences & Structural Analysis

Q1. Which preprocessing technique improved retrieval quality?

- Findings: Applying **Lowercasing** and **Stopword Removal** had the most positive impact on overall retrieval precision. Removing high-frequency structural noise (e.g., "the", "is", "of") prevents the coordinate matching logic from being misled by irrelevant documents, focusing search weights on high-entropy keywords.

Q2. Was stemming or lemmatization better for the dataset?

- Findings: **Lemmatization** outperformed stemming on the TREC collection. Porter's Stemmer strips semantic nuance, which often creates non-words (e.g., "computer" becomes "comput"). Snowball's lemmatization preserves stems, morphologically valid roots, which maintains downstream phrase integrity and improves dictionary lookups.

Q3. Which phrase query index was more accurate?

- Findings: The **Positional Index** provided perfect accuracy. The Inverted Index naively generates false positives because it only checks for adjacent token pairs, losing coherence over queries with 2 or more terms. The positional matrix verifies the structural order and exact spatial proximity of tokens across all sequences.

Q4. Which tree structure was faster?

- Findings: The **B-Tree** ($B=2$) outperformed the standard B-tree search. Because B-trees hold multiple keys per node and perform stable, balanced tree logic, their search depth is shallow. This prevents the repeated word resolution latency that affects standard B-trees when terms are indexed alphabetically.

Q5. How tolerant was their retrieval model?

- Findings: The inclusion of **Bigram** and **Trigram** expansion and a secondary **Lexical** database connection layer makes the system highly fault-tolerant against spelling mistakes and incomplete terms, ensuring smooth search execution even with imperfect user queries.

Q6. What are the current limitations of the system?

- Findings: Memory usage scales linearly because the system relies entirely on in-memory Python dictionaries. Furthermore, the search model uses unweighted boolean logic rather than a statistical, TF-IDF vector space framework with related cosine similarity.

Rocky LinuxRocky WSLRocky ForumRocky MailmanRocky Redd

Engine Global Controls

- Indexing
- Document Removal
- Hybrid Ranking

Report IssuesFeedback ComparisonPhrase ProcessingDictionary ToolsDocument RemovalIndexing Log

4. Performance Benchmarks: BST vs. B-Tree Dictionaries

Total Unique Vocabulary Entry Count: 4587 Terms

Successfully generated and memory-mapped BST and B-Tree $B=2$ structures.

Search Engine Latency Assessment Matrix

Evaluation Term / Prefix	BST Search Latency (ms)	B-Tree Search Latency (ms)	Index Speed Efficiency Factor
Information	0.00000	0.00000	0.00x faster
Index	0.00000	0.00000	0.00x faster
Computer	0.00000	0.00000	0.00x faster
Related	0.00000	0.00000	0.00x faster
Related	0.00000	0.00000	0.00x faster
File	0.00000	0.00000	0.00x faster
Library	0.00000	0.00000	0.00x faster
Research	0.00000	0.00000	0.00x faster
Sign	0.00000	0.00000	0.00x faster
Analysis	0.00000	0.00000	0.00x faster

Reference: The B-Tree structure maintains optimal structural balance with a predictable, consistent maximum node depth (O(log₂ N)). This significantly reduces tree traversal depth compared to an unbalanced Binary Search Tree (O(log₂ N)) or naive linear scans (O(N)), partially offsetting faster lookups in the dictionary size index.

Rocky LinuxRocky WSLRocky ForumRocky MailmanRocky Redd

Engine Global Controls

- Indexing
- Document Removal
- Hybrid Ranking

Report IssuesFeedback ComparisonPhrase ProcessingDictionary ToolsDocument RemovalIndexing Log

Advanced Information Retrieval System

5. Error Fault-Tolerance Verification Engine

Search Failure Mitigation Algorithm

When processing fails (e.g., `KeyError` or `ValueError`), the system will:

- 1. Check for typos in the query.
- 2. Check for missing stopwords.
- 3. Check for missing stopwords.
- 4. Check for missing stopwords.
- 5. Check for missing stopwords.
- 6. Check for missing stopwords.
- 7. Check for missing stopwords.
- 8. Check for missing stopwords.
- 9. Check for missing stopwords.
- 10. Check for missing stopwords.
- 11. Check for missing stopwords.
- 12. Check for missing stopwords.

Expanded Internal Core Terms Identified

* 1. "Information"

* 2. "Information"

* 3. "Information"

* 4. "Information"

* 5. "Information"

* 6. "Information"

* 7. "Information"

* 8. "Information"

* 9. "Information"

* 10. "Information"

* 11. "Information"

* 12. "Information"

Platform - Solutions - Resources - Open Source - Enterprise - Pricing

Search or jump to...

Sign inSign up

2025ab05012-lab / Group-74-IR-Assignment-1 Fork

Notifications Fork Star

CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

main - 2 branches 4 tags

Go to file

Code

2025ab05012-lab Improvements of home-update

Features 1 hour ago 3 Commits

Source

add files via upload

1 hour ago

README.md

Initial commit

1 hour ago

app.py

Add files via upload

1 hour ago

css_files.css

Add files via upload

1 hour ago

css_queries.css

Add files via upload

1 hour ago

css_util.css

Add files via upload

1 hour ago

prepare_package.py

Add files via upload

1 hour ago

requirements.txt

Requirements for native update

1 hour ago

README

Group-74-IR-Assignment-1

About

Group-74-IR-Assignment-1

Readme

Activity

1 star

1 watching

4 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors 1

2025ab05012-lab Forked Verónica N...